

Bike Path Detective

The City Ride Anomaly Hunter → turning messy mobility data into better city decisions

DRAGOMIR TIMEEA-ELENA

DRĂGHICI DENIS-DANIEL

MICEA CHRISTIAN-ANDREI



The Problem

Teleporting Bikes

Bikes appearing to cross town instantly, which are impossible ride records

Negative Durations

Rides with negative or zero minute durations are errors in the data

Double Bookings

One bike starting a second ride before finishing the first

Data Anomalies

Many other data irregularities and suspicious behavior



Who Feels the Pain?



End User: City Mobility Operations Analyst

Works for a public bike-sharing program. Needs to make real decisions about bikes, stations, and routes, but spends hours manually fixing spreadsheets instead.

Questions they need to answer

Which stations are busiest? Which routes are popular? Are any bikes producing impossible records? Which records should be excluded from reports?

How Bad Does It Hurt?

Estimates based on multiple reruns of our data generator - probabilities can easily be tuned

~10%

Records That Need Cleaning

Estimated share of ride records with formatting issues, duplicates, or inconsistencies

10

Known Stations

City network stations → easily adjustable and modifiable

~55%

Invalid Records

Estimated share with missing fields, impossible durations, invalid timestamps or other unfixable issues

10K

Ride Records

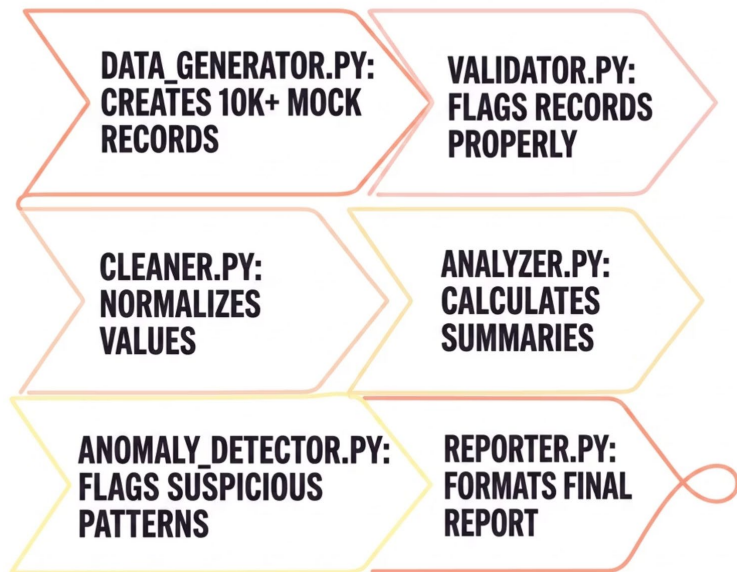
Dataset size → all requiring validation, cleaning, and anomaly detection

Our Technical Solution



 **Constraint:** Python 3.13 builtins + standard library only. No Pandas. No NumPy. No third-party packages.

Solution Architecture



Modular Design

Each module has a single responsibility. `main.py` orchestrates the full pipeline. Unit tests in `tests/` protect business logic.

- Data separated into **clean**, **needs-cleaning**, **suspicious** and **beyond-repair** groups
- Mock generator produces **10,000+ records** with controlled chaos
- Report written to `reports/...` text files and/or terminal

Validation & Cleaning Rules

What We Clean

- Trim whitespace, normalize casing
- Bike IDs → uppercase (bike-042 → BIKE-042)
- Station names → title case
- User types → lowercase
- Datetime formats unified
- Distance strings ("3.4 km" → 3.4)

What We Flag

- Missing required fields
- Duplicate ride IDs
- Unknown user types
- Unknown stations
- Negative or zero durations
- End time before start time
- Bikes with overlapping rides
- Duration mismatches vs. timestamps



Rule of thumb: Clean formatting problems. Report reality problems.

Anomaly Detection: 5+ Rules

1

Bike Overlapping

One bike in two rides at the same time

2

Station and Route Spikes

Station and Route usage $> \times 1.3$ average $\rightarrow$ event or data problem?

3

Duration $\neq$ Timestamps

Recorded duration differs from timestamp calculation within threshold

4

Unknown Stations

Unknown stations in the records could mean typos, data entry errors or system issues

5

Zero Duration Rides

Rides with duration = 0 that start and stop at the same station are OK, the rest are not

Technical Challenges We Faced



Separation of responsibilities



Generating realistic, varied data



Testing when working with 10K+ records



Repeated code with slight alterations



From Messy Records to Better Decisions

Identify

Unreliable ride records and impossible usage patterns

Locate

Overloaded stations and bikes with sync issues

Decide

Exclude bad data, act on real patterns with confidence